

SQL Practice Questions

BikeStores Dataset: Joins, GROUP BY, Subqueries & CTEs

Tables used

sales.orders	sales.order_items	sales.customers
sales.stores	sales.staffs	production.products
production.brands	production.categories	production.stocks

Order status codes: 1 = Pending, 2 = Processing, 3 = Rejected, 4 = Completed

Revenue formula: quantity * list_price * (1 - discount)

Joins

1. (Easy) List every order with the customer's full name, store name, and the full name of the staff member who handled it.

```
SELECT
    o.order_id,
    CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
    st.store_name,
    CONCAT(sf.first_name, ' ', sf.last_name) AS staff_name
FROM sales.orders o
JOIN sales.customers c ON o.customer_id = c.customer_id
JOIN sales.stores st ON o.store_id = st.store_id
JOIN sales.staffs sf ON o.staff_id = sf.staff_id;
```

2. (Easy) Show each product with its brand name and category name. Include products even if they have no brand or category assigned.

```
SELECT
    p.product_name,
    b.brand_name,
    c.category_name
FROM production.products p
LEFT JOIN production.brands b ON p.brand_id = b.brand_id
LEFT JOIN production.categories c ON p.category_id = c.category_id;
```

3. (Medium) Find all customers who have never placed an order. Return their name, city, and email.

```
SELECT
    c.first_name,
    c.last_name,
    c.city,
    c.email
FROM sales.customers c
LEFT JOIN sales.orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

GROUP BY

4. (Easy) Calculate total revenue per store. Revenue = quantity * list_price * (1 - discount). Sort from highest to lowest.

```
SELECT
    st.store_name,
    SUM(oi.quantity * oi.list_price * (1 - oi.discount)) AS total_revenue
FROM sales.stores st
JOIN sales.orders o ON st.store_id = o.store_id
JOIN sales.order_items oi ON o.order_id = oi.order_id
GROUP BY st.store_name
ORDER BY total_revenue DESC;
```

5. (Medium) For each brand, show the number of products, the average list price, and the highest list price. Only include brands with more than 5 products.

```
SELECT
    b.brand_name,
    COUNT(p.product_id) AS product_count,
    AVG(p.list_price) AS avg_price,
    MAX(p.list_price) AS max_price
FROM production.brands b
JOIN production.products p ON b.brand_id = p.brand_id
GROUP BY b.brand_name
HAVING COUNT(p.product_id) > 5;
```

6. (Medium) Show the number of orders and total revenue per month for the year 2017, ordered chronologically.

```
SELECT
    MONTH(o.order_date) AS order_month,
    COUNT(DISTINCT o.order_id) AS num_orders,
    SUM(oi.quantity * oi.list_price * (1 - oi.discount)) AS total_revenue
FROM sales.orders o
JOIN sales.order_items oi ON o.order_id = oi.order_id
WHERE YEAR(o.order_date) = 2017
GROUP BY MONTH(o.order_date)
ORDER BY order_month;
```

Subqueries

7. (Medium) Find all products priced above the average list price of their own category.

Hint: Use a correlated subquery.

```
SELECT
    p1.product_name,
    p1.list_price,
    p1.category_id
FROM production.products p1
WHERE p1.list_price > (
    SELECT AVG(p2.list_price)
    FROM production.products p2
    WHERE p2.category_id = p1.category_id
);
```

8. (Medium) List the customers who have placed more orders than the average number of orders per customer.

```
SELECT
    c.customer_id,
    c.first_name,
    c.last_name,
    COUNT(o.order_id) AS order_count
FROM sales.customers c
```

```

JOIN sales.orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.first_name, c.last_name
HAVING COUNT(o.order_id) > (
    SELECT AVG(order_count)
    FROM (
        SELECT COUNT(order_id) AS order_count
        FROM sales.orders
        GROUP BY customer_id
    ) AS avg_table
);

```

CTEs

9. (Hard) Using a CTE, calculate each customer's total spend, then return the top 10 customers with their spend and rank. Add a second CTE that labels each customer as "High" (above the overall average spend) or "Regular".

```

WITH CustomerSpend AS (
    SELECT
        c.customer_id,
        CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
        SUM(oi.quantity * oi.list_price * (1 - oi.discount)) AS total_spend
    FROM sales.customers c
    JOIN sales.orders o ON c.customer_id = o.customer_id
    JOIN sales.order_items oi ON o.order_id = oi.order_id
    GROUP BY c.customer_id, c.first_name, c.last_name
),
OverallAvg AS (
    SELECT AVG(total_spend) AS avg_spend FROM CustomerSpend
)
SELECT TOP 10
    cs.customer_name,
    cs.total_spend,
    RANK() OVER(ORDER BY cs.total_spend DESC) AS spend_rank,
    CASE

```

```

        WHEN cs.total_spend > oa.avg_spend THEN 'High'
        ELSE 'Regular'
    END AS customer_status
FROM CustomerSpend cs
CROSS JOIN OverallAvg oa
ORDER BY cs.total_spend DESC;

```

10. (Hard) Using CTEs, find the best-selling product (by quantity) in each category, and show how much of that product's stock is currently available across all stores.

Hint: Use ROW_NUMBER() or RANK() partitioned by category, then join to production.stocks.

```

WITH ProductSales AS (
    SELECT
        p.category_id,
        p.product_id,
        p.product_name,
        SUM(oi.quantity) AS total_sold
    FROM production.products p
    JOIN sales.order_items oi ON p.product_id = oi.product_id
    GROUP BY p.category_id, p.product_id, p.product_name
),
RankedSales AS (
    SELECT
        category_id,
        product_id,
        product_name,
        total_sold,
        ROW_NUMBER() OVER(PARTITION BY category_id ORDER BY total_sold DESC) AS rn
    FROM ProductSales
)
SELECT
    rs.category_id,
    rs.product_name,
    rs.total_sold,
    COALESCE(SUM(s.quantity), 0) AS total_stock
FROM RankedSales rs
LEFT JOIN production.stocks s ON rs.product_id = s.product_id
WHERE rs.rn = 1

```

GROUP BY rs.category_id, rs.product_name, rs.total_sold

Bonus Challenges

- Rewrite Q8 using a CTE instead of a subquery and compare readability.
- For Q4, add a column showing each store's percentage share of total company revenue.